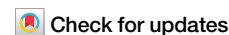


<https://doi.org/10.1038/s43246-025-00994-x>

Modular large language model agents for multi-task computational materials science

Akshat Chaudhari¹, Janghoon Ock^{2,3} & Amir Barati Farimani^{2,4,5,6}

The integration of large language models (LLMs) with domain-specific computational tools provides a pathway to streamline and enhance materials science workflows. This paper introduces MatSciAgent, a multi-agent framework supporting tasks such as materials data retrieval, continuum simulation, crystal structure generation, and molecular dynamics simulation. At its core is a master agent that interprets user queries, identifies the task type, and delegates to task-specific agent(s) equipped with tools. Leveraging databases such as Materials Project and MatWeb, the framework retrieves and summarizes materials data with grounded, factual responses, addressing limitations of vanilla LLMs. When a target material is absent from databases, a generative agent can propose plausible crystal structures. For simulations, specialized agents extract parameters to perform continuum and molecular dynamics simulations using existing software or custom code. MatSciAgent demonstrates stability, with parameter extraction achieving 100% success across five runs and materials extraction consistent in 9 of 10 runs. Its modular design ensures seamless extensibility to evolve as new capabilities are integrated.

Materials science and engineering are central to technological progress, enabling the discovery, design, and optimization of materials tailored to specific performance requirements. Emerging industries such as aerospace, energy, and biotechnology increasingly demand materials with precisely engineered mechanical, thermal, and electrical properties. Designing such materials is a complex and iterative process that requires deep domain expertise and advanced computational techniques. As a result, researchers are increasingly leveraging computational tools to predict and optimize material behavior, accelerating the development of materials with targeted functionalities^{1,2}.

This shift has driven the creation of large-scale materials databases including the Materials Project³, MatWeb⁴, OQMD⁵, and ICSD⁶—which serve as critical resources for data-driven materials discovery. Predictive models built on these databases are increasingly used to guide materials design decisions⁷. In particular, generative models—machine learning algorithms that learn patterns from data to generate new, plausible candidates are being adopted to propose material compositions or structures, further accelerating discovery^{8–10}. However, despite these advancements, integrating various simulation tools and data sources into a unified framework

remains a significant challenge. This lack of integration limits the seamless use of computational resources and hinders the pace of innovation in materials design¹¹.

Recent advances in artificial intelligence—especially the emergence of large language models (LLMs) have reshaped how researchers engage with complex data and derive scientific insights. Built on transformer architectures¹², these models excel at understanding and generating natural language through self-attention mechanisms, enabling scalable processing of language-based information. LLMs, such as OpenAI's GPT models^{13,14}, Meta's LLaMA¹⁵, and Anthropic's Claude¹⁶, have demonstrated exceptional capabilities in interpreting user queries, synthesizing information from diverse sources, and producing actionable insights. In the context of materials science, LLMs support tasks such as information extraction from large databases, literature mining, and hypothesis generation. Their ability to process unstructured natural language queries allows researchers to intuitively explore complex datasets, reducing manual effort and accelerating scientific workflows^{17–19}. Additionally, transformer-based language models have proven effective as encoders for property prediction tasks across a range of scientific domains, including metal-organic frameworks²⁰,

¹Department of Material Science and Engineering, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, USA. ²Department of Chemical Engineering, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, USA. ³Department of Chemical and Biomolecular Engineering, University of Nebraska–Lincoln, Lincoln, NE, USA. ⁴Department of Mechanical Engineering, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, USA. ⁵Department of Biomedical Engineering, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, USA. ⁶Machine Learning Department, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, USA. e-mail: jock2@unl.edu; barati@cmu.edu

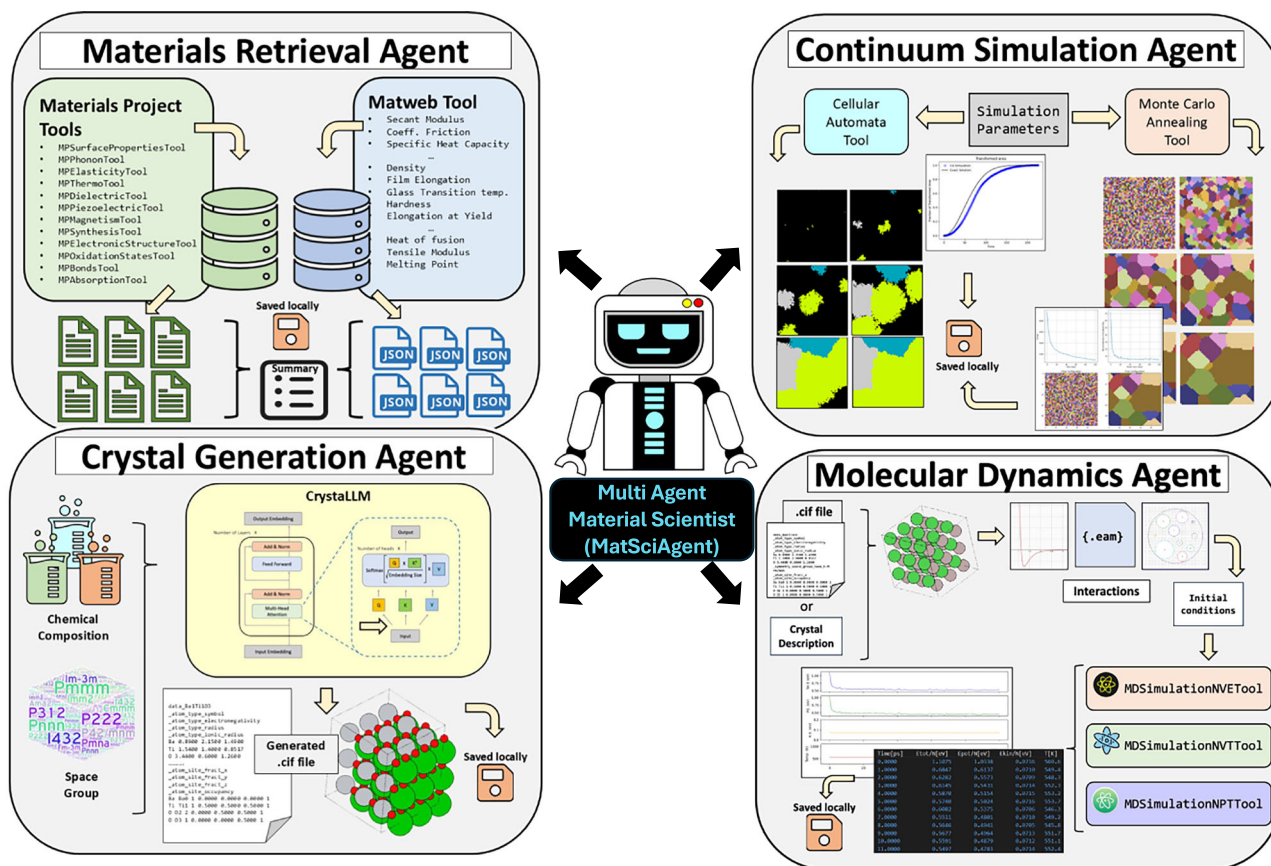


Fig. 1 | MatSciAgent architecture. This illustration showcases how each agent leverages input from the user, integrates with domain-specific tools, and utilizes the reasoning capabilities of the LLM to process tasks and generate outputs. OpenAI's GPT-3.5-turbo serves as the core LLM in this framework.

polymers²¹, catalysis^{18,19}, proteins^{22–25}, inorganic crystals^{26,27}, and additive manufacturing²⁸.

LLMs are now being integrated into agentic frameworks that extend their utility from passive query interpretation to active scientific problem-solving^{29,30}. These LLM agents leverage the model's language understanding, pre-trained knowledge, and emerging planning abilities to autonomously perform complex tasks. For instance, Boiko et al. introduced Coscientist, an LLM-driven agent that automates experimental design and execution, substantially reducing manual effort³¹. Similarly, ChemCrow combines LLMs with chemistry-specific tools to support synthesis planning and materials discovery³². Such agentic systems are being applied across diverse scientific domains, including not only materials science and chemistry but also fields like 3D printing and engineering design^{33,34}.

While LLMs demonstrate strong capabilities in natural language processing and general reasoning, their application in specialized scientific domains such as materials science presents several challenges. One of the most significant concerns is hallucination, where LLMs produce information that appears plausible but is factually incorrect^{35–37}. Such outputs can mislead researchers, disrupt workflows, and lead to wasted resources. This issue is particularly problematic when interpreting technical datasets, as LLMs are typically trained on general-purpose corpora that lack sufficient coverage of domain-specific materials science knowledge^{38,39}.

Furthermore, LLMs alone are inherently limited in delivering rigorous scientific predictions for complex physical phenomena, as such tasks require high-precision, physics-based simulations and validated numerical methods. To overcome this limitation, LLMs must be integrated with domain-specific scientific software, enabling them to function as user

proxies that automate the setup and execution of simulation workflows. Several hybrid frameworks have emerged to support this vision—LLaMP⁴⁰, HoneyComb⁴¹, and AtomAgents⁴² which couple LLMs with scientific tools and databases. LLaMP and HoneyComb primarily function as retrieval agents, leveraging data sources like the Materials Project, arXiv, and Wikipedia. AtomAgents is a specialized intelligent system for alloy development, integrating multimodal perception (e.g., microstructure images, compositional data, phase diagrams) with domain-specific reasoning and materials design tools. Its core functionality lies in generating and optimizing alloy compositions, predicting properties, and suggesting processing pathways tailored towards performance requirements. As research continues, there is a growing need for a more generalizable and modular framework that can accommodate a broader range of materials systems and simulation tasks, while remaining flexible enough to incorporate newly developed computational tools.

We propose MatSciAgent, a multi-agent framework that integrates an LLM with specialized computational tools to perform tasks based on natural language user queries. As illustrated in Fig. 1, the system supports four task types: materials retrieval, continuum simulation, crystal structure generation, and molecular dynamics simulation. Each task is handled by a dedicated task-specific agent, coordinated by a central master agent. The master agent interprets the user's input, determines the type of task, and delegates the query to the appropriate task-specific agent. The assigned agent then selects the relevant computational tools needed to complete the task. All decisions, from task classification to tool selection, are guided by the reasoning capabilities of the LLM. This framework demonstrates how LLMs can enhance computational materials science by automating workflows, improving accessibility to simulation tools, and accelerating research processes.

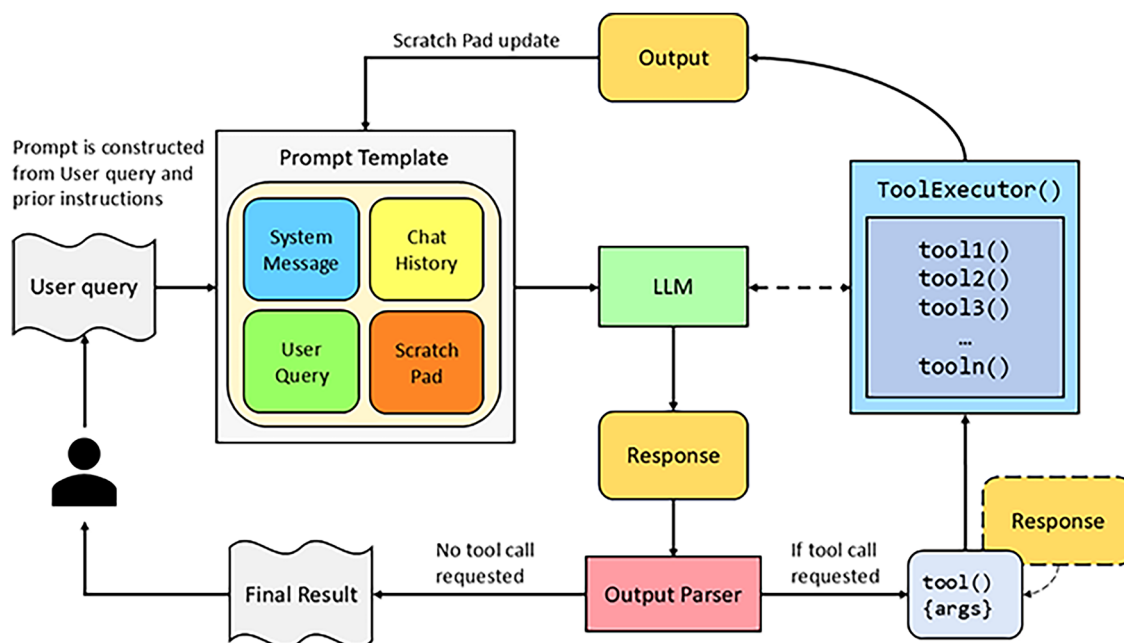


Fig. 2 | Components of an LLM agent. User input is structured using a prompt template that includes a system message, dialogue history, user query, and a scratchpad for intermediate results. The LLM processes the prompt, and an output parser determines whether a tool invocation is needed. If so, the tool execution

engine runs the specified tool and updates the scratchpad with the results. This updated scratchpad is reintegrated into the prompt template for iterative processing. The final result is produced once no further tool invocations are required.

Results and discussions

Multi-agent system

The workflow consists of two key steps: First, when a user submits a query, a master agent classifies the task type and routes the query to the appropriate task-specific agent. Then, the task-specific agent selects and deploys the most suitable computational tools to solve the problem, delivering results back to the user. Built on OpenAI's GPT-3.5-turbo, it employs customized prompts for domain-specific reasoning and tool integration. The modular architecture supports diverse tasks, including data extraction, continuum simulations, crystal structure generation, and molecular dynamics (MD) simulations, ensuring easy expansion with new agents, tools, or workflows.

Each individual agent is powered by an LLM, equipped with task-specific tools, and operates with shared memory supported by a scratchpad, as shown in Fig. 2. A key strength is its agentic design, where each agent operates independently and is equipped with specialized tools and prompts. The scratchpad mechanism facilitates information exchange process within the framework. When a user query enters the system, the MatSciAgent selects the appropriate agent to handle the request. Initially empty, the scratchpad begins recording the agent's reasoning and actions. As the agent identifies a need for specific information, it invokes appropriate tools and these invocations are added to intermediate steps. The tool executes and returns results, which are then formatted and pushed back into the scratchpad. Each time the agent is called, it receives this updated scratchpad containing all previous tool calls and results, enabling continuous, coherent reasoning. This process allows the agent to progressively build knowledge by tracking previous findings, connecting different pieces of information, and maintaining context throughout multi-step workflows. The agent continues this cycle until it has gathered sufficient information to form a final response, which is then extracted and presented to the user. The LLM serves as the reasoning layer, dynamically interpreting queries and orchestrating tool use. By combining LLM-driven reasoning with the precision of computation tools, the system can handle complex queries, perform precise calculations, and generate analyzable insights.

Prompt engineering and structured tool integration enhance adaptability. Carefully designed prompts define each agent's operations, while integrated tools handle specialized computations like MD simulations or

property data extraction. The LLM remains focused on reasoning and decision-making while delegating computationally intensive operations to these tools. A centralized execution framework manages input parsing, tool invocation, and response formatting, ensuring seamless interaction between the LLM's reasoning and the tools' deterministic computations. For details about the individual agents, kindly refer to the Methods section.

Materials extraction | Case 1. Synthesis of perovskite oxides for solar cells

The task begins with the user query: "Provide synthesis details for perovskite oxides used in solar cells." (Full prompt and response shown in Supplementary Information Case 1.) A comparison between the materials extraction agent and the vanilla GPT-3.5-turbo is summarized in Fig. 3. The materials extraction agent first identifies key terms from the query namely "perovskite" and "solar cells" using its reasoning capabilities. It then invokes the materialsproject API to retrieve relevant data from the database. This approach enables the agent to deliver more specific and detailed information than the vanilla GPT-3.5-turbo. For example, it identifies specific compositions such as $\text{La}_{0.8}\text{Sr}_{0.2}\text{Co}_x\text{Fe}_{1-x}\text{O}_3$ and provides corresponding precursor details. It also outlines precise synthesis protocols, including temperature conditions and duration. In contrast, GPT-3.5-turbo returns only generic information and fails to offer detailed synthesis procedures.

Materials extraction | Case 2. Bonding information for TiO_2

The user input query in this task specifies a particular material and requested properties, such as: "Retrieve bond length and coordination information for TiO_2 . List your results." (Full prompt and response shown in Supplementary Information Case 2.) The agent first identifies the task as a material/property extraction query and summarizes the retrieved information, as shown in Fig. 4. In this case, the target material is TiO_2 , and the properties of interest are bond lengths and coordination environments. Because TiO_2 exists in multiple structural forms each associated with a unique material ID the agent begins by retrieving all relevant material IDs. It then uses a set of tools to extract the bonding properties for each structure.

(a) User Query

Provide synthesis details for perovskite oxides used in solar cells.

(b) Materials Extraction Agent

Invoking: 'MaterialsProjectSynthesis' with {'keywords': ['perovskite', 'solar cells'], 'num_chunks': 1, 'chunk_size': 10}
 responded: This query falls under problem type '1' for finding materials based on physical properties. To provide synthesis details for perovskite oxides used in solar cells, I will first search for materials based on the given criteria using the appropriate tool. Let's proceed with searching for perovskite oxides used in solar cells.

The study focused on synthesizing $\text{La}_{0.8}\text{Sr}_{0.2}\text{Co}_{0.5}\text{Fe}_{0.5}\text{O}_{3-x}$ via a solid-state method using specific precursors and a complex reaction involving metal nitrates and oxides. The fuel cells were made with YSZ scaffolds. The synthesis process included mixing and heating operations with precise conditions such as calcination at temperatures ranging from 723 K to 1373 K. ...

The study focused on synthesizing $\text{La}_{0.8}\text{Sr}_{0.2}\text{CoO}_3$ using a solid-state method with specific precursor materials including $\text{Sr}(\text{NO}_3)_2$, $\text{La}(\text{NO}_3)_3$, and $\text{Co}(\text{NO}_3)_3$. The synthesis involved a reaction resulting in $\text{La}_{0.8}\text{Sr}_{0.2}\text{CoO}_3$ formation along with by-products like H_2O and $[\text{NO}_3^-]$. Key operations included fabrication and calcination steps at different temperatures (723K and 1123-1373K) to yield the target material. ...

The study focused on synthesizing $\text{Cu}_2\text{ZnSnSe}_4$ (CZTSe) through solid-state reaction. Key findings include the composition of CZTSe with 2Cu, 1Zn, 1Sn, and 4Se atoms. The synthesis involved mixing Cu, Zn, Sn, and Se precursors, followed by heating at 300°C for 30 minutes in a N_2 atmosphere. Solar cells based on CZTSe were fabricated, demonstrating potential for photovoltaic applications.

...
 The study presented solid-state synthesis of $\text{Cu}_2\text{Zn}_{1.5}\text{SnSe}_4$ via the reaction $2\text{Cu} + 4\text{Se} + 1\text{Sn} + 1.5\text{Zn} \Rightarrow 1\text{Cu}_2\text{Zn}_{1.5}\text{SnSe}_4$. The process involved starting synthesis, mixing, drying, pressing, and firing at 300°C for 30 minutes in an N_2 atmosphere.

...
 ...Here are some key findings:

- 1. $\text{La}_{0.8}\text{Sr}_{0.2}\text{Co}_{0.5}\text{Fe}_{0.5}\text{O}_{3-x}$ Synthesis:**
 - Synthesized via a solid-state method using specific precursors and a complex reaction involving metal nitrates and oxides.
 - Fabrication of fuel cells with YSZ scaffolds.
 - Mixing and heating operations with precise conditions such as calcination at temperatures ranging from 723 K to 1373 K.
- 2. $\text{La}_{0.8}\text{Sr}_{0.2}\text{FeO}_3$ Synthesis:**
 - Solid-state synthesis using specific precursors including $\text{Fe}(\text{NO}_3)_3 \cdot 9\text{H}_2\text{O}$, $\text{La}(\text{NO}_3)_3 \cdot 6\text{H}_2\text{O}$, O_2 , $\text{Sr}(\text{NO}_3)_2$, and $[\text{OH}^-]$.
 - Fabrication of fuel cells with porous-dense-porous YSZ scaffolds.
 - Mixing and heating operations with calcination at temperatures ranging from 723.0 K to 1373.0 K.
- 3. $\text{Cu}_2\text{ZnSnSe}_4$ (CZTSe) Synthesis:**
 - Solid-state reaction involving Cu, Zn, Sn, and Se precursors.
 - Fabrication of solar cells with an active area of $\sim 0.25 \text{ cm}^2$.
 - Mixing, drying, pressing, and firing at 300°C for 30 minutes in a N_2 atmosphere.

(c) GPT-3.5-turbo

Perovskite oxides are a class of materials with the general chemical formula ABX_3 , where A and B are cations and X is an anion. These materials have gained significant attention in the field of solar cells due to their excellent optoelectronic properties, such as high carrier mobility, tunable band gap, and high absorption coefficients.

One common synthesis method for perovskite oxides involves the **solid-state reaction** technique. In this method, the precursors containing the A, B, and X elements are mixed in appropriate stoichiometric ratios and then heated at high temperatures (typically above 600°C) to facilitate the formation of the perovskite crystal structure. The reaction can be carried out in air or under controlled atmosphere conditions to prevent oxidation or other unwanted reactions.

Another widely used method for synthesizing perovskite oxides is the **sol-gel technique**. In this method, metal alkoxides or other metal precursors are dissolved in a solvent to form a sol. This sol is then subjected to hydrolysis and condensation reactions to form a gel, which is subsequently dried and calcined to obtain the perovskite oxide material. This method allows for better control over the stoichiometry and morphology of the final material.

Other techniques such as **chemical vapor deposition**, **hydrothermal synthesis**, and **co-precipitation** have also been employed for the synthesis of perovskite oxides used in solar cells. These methods offer advantages such as higher purity, better crystallinity, and tunable properties of the final material.

Overall, the synthesis of perovskite oxides for use in solar cells requires careful control of the reaction conditions, stoichiometry, and processing parameters to achieve the desired optoelectronic properties. The choice of synthesis method depends on factors such as the specific perovskite composition, desired properties, scalability, and cost-effectiveness.

Fig. 3 | Synthesis details generation. **a** User query specifying the intended retrieval objective. **b** Output from the materials extraction agent (full prompt in Supporting Information). **c** Output from GPT-3.5-turbo.

Because the agent has access to the Materials Project database, it generates response prompts based on real, up-to-date data. In contrast, the vanilla GPT-3.5-turbo relies solely on its pre-trained knowledge, without access to external sources. Although only one summary is shown below for illustration, the agent produces a separate summary for each retrieved structure, ensuring thorough coverage of all known structural variants of the material.

Materials extraction | Case 3: Materials within a specific heat capacity range

The user query specifies a target property range—specific heat capacities between 0.1 and 2.0 J/g · °C—along with additional guidelines, such as ensuring material diversity and prioritizing experimentally verified data (Fig. 5a) (Full prompt and response shown in Supplementary Information Case 3). The agent begins by identifying the property of interest and its specified range, then queries the MatWeb database to retrieve relevant

materials and generates a structured response (Fig. 5b). For comparison, Fig. 5c shows the output from the vanilla GPT-3.5-turbo for the same input.

The agent identifies example materials within the specified heat capacity range and supplements this with additional thermal, mechanical, and compositional properties. For instance, silver (Ag) is retrieved with a specific heat capacity of 0.234 J/g · °C, along with a density of 10.491 g/cc, thermal conductivity of 419 W/m · K, and melting point of 961.93°C. Aluminum (Al), with a specific heat capacity of 0.900 J/g · °C, is also returned, featuring a low density of 2.6989 g/cc and thermal conductivity of 210 W/m · K. Additionally, the agent includes argon (Ar), a gas with a specific heat capacity of 0.520 J/g · °C, demonstrating its ability to handle a wide range of material types.

In contrast, the vanilla GPT-3.5-turbo can identify materials that fall within the specified heat capacity range but fails to provide accompanying

(a) User Query

Retrieve bond length and coordination information for TiO₂. List your results.

(b) Materials Extraction Agent

Invoking: 'get_material_ids' with '{'compound': 'TiO2'}' responded: This query falls under Type 2, which involves getting properties of a specific material. The specified material is TiO₂, which corresponds to the chemical formula of Titanium Dioxide. To proceed, I will first need to obtain the material_id(s) for TiO₂ by using the chemical formula. After that, I will extract the bond length and coordination information for TiO₂. Let's start by retrieving the material_id(s) for TiO₂. [MPID(mp-390), MPID(mp-430), MPID(mp-1439), MPID(mp-1840), MPID(mp-2657), MPID(mp-9173), MPID(mp-25262), MPID(mp-25433), MPID(mp-34688), MPID(mp-553432), MPID(mp-554278), MPID(mp-572822), MPID(mp-636827), MPID(mp-655656), MPID(mp-754672), MPID(mp-754769), MPID(mp-756587), MPID(mp-756655), MPID(mp-766454), MPID(mp-775938), MPID(mp-777129), MPID(mp-849533), MPID(mp-1008677), MPID(mp-1041565), MPID(mp-1044853), MPID(mp-1079207), MPID(mp-1101022), MPID(mp-1102255), MPID(mp-1102591), MPID(mp-1190753), MPID(mp-1201727), MPID(mp-1245098), MPID(mp-1245134), MPID(mp-1245308), MPID(mp-1245321), MPID(mp-1341203), MPID(mp-1350429), MPID(mp-1380335), MPID(mp-1386640), MPID(mp-1400963), MPID(mp-1408285), MPID(mp-1426806), MPID(mp-1445035), MPID(mp-2420244), MPID(mp-2739239), MPID(mp-2901430)]...

```
1. **Material Properties**:
  - **Crystal System**: Orthorhombic
  - **Volume**: Ranges from 113.66 Å3 to 757.26 Å3
  - **Density**: Ranges from 1.40 g/cm3 to 4.56 g/cm3
2. **Bonding Information**:
  - **Coordination Environments**: O2--Ti4+(3) and Ti4--O2-(6)
  - **Bond Lengths**: Range from 1.88 Å to 2.20 Å
  - **Bond Types**: O2--Ti4+ interactions 3. **Crystal Structure**:
  - **Space Group**: Pbcn, Pbca, Fd-3m, P4_2/mnm, C2/c, I4_1/amd, Pnma, C2/m, P3_121, Fd-3m, Pmm2, Cmm2, Pnma
4. **Material IDs**:
  - mp-390, mp-430, mp-1439, mp-1840, mp-2657, mp-9173, mp-25262, mp-25433, mp-34688, mp-553432, mp-554278, mp-572822, mp-636827, mp-655656, mp-754672, mp-754769, mp-756587, mp-756655, mp-766454, mp-775938, mp-777129, mp-849533, mp-1008677, mp-1041565, mp-1044853, mp-1079207, mp-1101022, mp-1102255, mp-1102591, mp-1190753, mp-1201727, mp-1245098, mp-1245134, mp-1245308, mp-1245321, mp-1341203, mp-1350429, mp-1380335, mp-1386640, mp-1400963, mp-1408285, mp-1426806, mp-1445035, mp-2420244, mp-2739239, mp-2901430
```

(c) GPT-3.5-turbo

In titanium dioxide (TiO₂), the bond length of the Ti-O bond is approximately 1.95 Å (angstroms).
In terms of coordination, Ti in TiO₂ typically exhibits octahedral coordination, where each titanium atom is surrounded by six oxygen atoms arranged in an octahedral fashion.

Fig. 4 | Material-specific information retrieval. **a** User query specifying the target material. **b** Output from the materials extraction agent (full prompt in Supporting Information). **c** Output from GPT-3.5-turbo.

property data. This omission would require additional searches, reducing its overall utility for comprehensive materials analysis.

Continuum simulation | Case 1: Al-Cu-Mg alloy solidification with diffusion-controlled growth

The user provides a query specifying the desired simulation conditions, as illustrated in Fig. 6a. The agent interprets the input, extracts actionable simulation parameters such as: width, attachment probability, and the number of solid phases; and selects the appropriate tool—in this case, the CA tool for simulating phase transformations. It then runs the CA simulation using the derived parameters and generates a summary of the results, including a visualization of the phase diagram. The evolution of the microstructure is captured frame by frame, as shown in Fig. 6b, highlighting randomly seeded nuclei distributed across the domain and forming dendritic growth patterns typical of diffusion-controlled solidification.

Continuum simulation | Case 2: Ti-Al intermetallic alloy solidification with interface-controlled growth

In this case study, the continuum simulation agent is applied to model interface-controlled growth in an alloy system. The agent interprets the user's natural language query, extracts the relevant simulation parameters, and executes a CA simulation to generate frame-by-frame images of microstructural evolution during solidification, as shown in Fig. 7. The simulation results reveal smooth, isotropic growth behavior, characterized by uniformly expanding grains without the formation of dendritic arms—consistent with the high attachment probability conditions. Controlled nucleation, set at 50 nuclei per unit area, leads to evenly distributed grains across the domain, resulting in a homogeneous microstructure. The high attachment probability also enhances solid-liquid interface kinetics, promoting the formation of uniformly sized grains with minimal solute segregation.

Continuum simulation | Case 3: Polycrystalline grain growth using Monte Carlo annealing

This task involves performing an MCA simulation to model polycrystalline grain growth as shown in Fig. 8. Unlike the previous two case studies, the agent applies the MCA simulation tool based on the description provided in the user query. The simulation, conducted on a 50 × 50 lattice grid, demonstrates classic grain coarsening behavior—larger grains grow at the expense of smaller ones. The lattice initially contains randomly oriented grains, but over the course of 100 Monte Carlo steps, the system evolves to minimize interfacial energy. Grain boundaries shift, especially at interfaces where different grains meet, driving the reduction of total grain boundary energy. This results in a coarser microstructure, characterized by fewer, larger grains—consistent with the typical grain growth behavior observed during the annealing of polycrystalline materials.

Crystal generation | Case 1: CIF for alloy

The task begins with a user query specifying the chemical formula and space group of a crystal material (Fig. 9(a)). The agent interprets this prompt and extracts the relevant crystallographic parameters—in this case, the composition of the Ni₃Al alloy and the space group *Pm* $\bar{3}$ *m*. Using CrystalLM⁸ as a tool, the agent generates a corresponding CIF file, which describes the crystal structure visualized in Fig. 9(b). Based on the CIF file, the agent constructs a natural language response that includes precise lattice parameters ($a = b = c = 3.5555$) and atomic positions (Al at (0, 0, 0) and Ni at (0, 0.5, 0.5)), consistent with the specified symmetry and stoichiometry. In addition to atomic coordinates, the CIF file also contains elemental properties such as electronegativity and ionic radii.

(a) User Query

Identify and extract materials with a specific heat capacity in the range of 0.1 to 2.0 Joules per gram per degree Celsius from the dataset. Ensure the retrieved materials represent diverse material classes, including metals, polymers, ceramics, and composites. For each material, provide its specific heat capacity value along with additional thermal, mechanical, and compositional properties, as available. Prioritize entries with experimentally measured data, clearly indicating the temperature conditions under which the specific heat capacity was determined. Exclude materials with estimated or incomplete data, and present the results in a structured format for comparative analysis.

(b) Materials Extraction Agent

Invoking: `extract\_materials\_matweb` with `{ 'property': 'Specific Heat Capacity (J/g°C)', 'low': 0.1, 'high': 2.0, 'num\_materials': 5 }` responded: This query falls under Type 1: Finding materials based on physical properties. The user is requesting the identification and extraction of materials with a specific heat capacity in the range of 0.1 to 2.0 Joules per gram per degree Celsius from the dataset. The user also specified the need for diverse material classes, including metals, polymers, ceramics, and composites, and additional thermal, mechanical, and compositional properties for each material.

```
...
1. **Silver (Ag)**
  - Specific Heat Capacity: 0.520 J/g-°C
  - Density: 10.491 g/cc
  - Melting Point: 961.93°C
  - Boiling Point: 2162°C
  - Thermal Conductivity: 419 W/m-K
  - Electrical Resistivity: 0.0000155 ohm-cm
  - Vickers Hardness: 25
  - Tensile Strength: 140 MPa
  - Heat of Fusion: 105 J/g
  - Heat of Vaporization: 2332 J/g

2. **Aluminum (Al)**
  - Specific Heat Capacity: 0.897 J/g-°C
  - Density: 2.6989 g/cc - Melting Point: 660.37°C
  - Boiling Point: 2519°C
  - Thermal Conductivity: 210 W/m-K
  - Electrical Resistivity: 0.0000270 ohm-cm
  - Vickers Hardness: 15
  - Modulus of Elasticity: 68.0 GPa
  - Heat of Fusion: 386.9 J/g
  - Heat of Vaporization: 9462 J/g

3. **Argon (Ar)**
  - Specific Heat Capacity: 0.520 J/g-°C
  - Density: 0.001633 g/cc
  - Melting Point: -189.35°C
  - Boiling Point: -185.85°C
  - Thermal Conductivity: 0.0102 W/m-K
  - Heat of Fusion: 29.54 J/g
  - Heat of Vaporization: 161.0 J/g
...
```

(c) GPT-3.5-turbo

Material	Specific Heat Capacity (J/g°C)	Additional Properties	Source
Copper	0.39 (at 25°C)	High thermal conductivity, good electrical conductivity	Experimentally measured at 25°C
Aluminum	0.897 (at 25°C)	Lightweight, corrosion resistance	Experimentally measured at 25°C
Polyethylene	2.3 (at 25°C)	Low density, good chemical resistance	Experimentally measured at 25°C
Titanium	0.523 (at 25°C)	High strength-to-weight ratio, corrosion resistance	Experimentally measured at 25°C
Glass	0.84 (at 25°C)	Brittle, transparent, insulating properties	Experimentally measured at 25°C
Carbon Fiber Composite	~1.12 (at 25°C)	High strength, low weight	Experimentally measured, exact conditions unavailable
Water	4.18 (at 25°C)	High heat capacity, universal solvent	Experimentally measured at 25°C

Fig. 5 | Materials extraction based on target property range. **a** User query specifying the desired property range. **b** Output from the materials extraction agent (full prompt in Supporting Information). **c** Output from GPT-3.5-turbo.

Crystal Generation | Case 2: CIF for arbitrary compounds

For an arbitrary material, $\text{Sr}_2\text{FeMoO}_6$ in the double perovskite structure, the agent demonstrated its ability to translate natural language-based queries into crystallographic description, as shown in Fig. 10. The resulting CIF file provides structural details, including lattice parameters ($a = b = 5.65 \text{ \AA}$, $c = 3.9733 \text{ \AA}$, $\alpha = \beta = \gamma = 90^\circ$) and atomic positions for Sr, Fe, Mo, and O within the specified space group $P4/mmm$. This highlights the agent's ability to address complex and arbitrary material compositions.

Molecular dynamics | Case 1: MD simulation of aluminum at constant energy

In this case, the molecular dynamics agent performs an MD simulation of aluminum (Al). It interprets the natural language query to extract simulation conditions, identifying a constant energy (NVE) ensemble with a target temperature of 660°C , which it correctly converts to 933.15 K , as illustrated in Fig. 11.

The system is initialized using a face-centered cubic (FCC) crystal structure with a lattice constant of $4.05 \text{ \AA}$. Interatomic interactions are modeled using the Effective Medium Theory (EMT) potential, as specified in the query. The simulation runs for 100,000 steps (Fig. 11b). The results show that the system's total energy remains constant throughout the simulation, consistent with the expected behavior of an NVE ensemble. The temperature fluctuates around the target value, with kinetic and potential energy components varying over time. During the simulation, the

temperature stabilizes within a range of $\sim 471 \text{ K}$ – 685 K . The corresponding MD trajectory can be found in Supplementary Data 1.

Joint Implementation

To demonstrate the capability of our LLM agent system to coordinate multiple domain-specialized agents for a multi-step materials science task, we designed a two-stage experiment involving crystal structure generation followed by MD simulation as shown in Fig. 12. The first stage involved a user prompt requesting a CIF for the intermetallic compound NiAl in the cubic Pm-3m space group. MatSciAgent, correctly delegated the task to the CrystalGenerationAgent, which is responsible for generating crystal structures. This agent invoked the CrystaLLM tool with the specified parameters (composition = 'Ni₁Al₁', space group = 'Pm-3m') and successfully produced the corresponding CIF file.

In the second stage, a follow-up prompt requested an MD simulation of the generated AlNi structure under NVT conditions, using the EMT potential and a Langevin thermostat set to an initial temperature of 550 K for 100,000 steps. The corresponding MD trajectory can be found in Supplementary Data 2. The system again selected the appropriate agent MolecularDynamicsAgent which used the MDSimulationNVTTool to carry out the simulation, automatically extracting relevant parameters from the previously generated CIF file.

This experiment illustrates the system's ability to preserve context across agent transitions and demonstrates seamless operation between

(a) User Query

Simulate the solidification of an Al-Cu-Mg alloy system using a CA model. The system should be modeled on a square domain with a width of 100 grid units to ensure adequate resolution of microstructural features and multi-phase interactions. Set the attachment probability to 0.2, representing realistic diffusion-controlled growth processes for metallic systems under typical cooling rates.

(b) Continuum Simulation Agent

> Entering new AgentExecutor chain...

Invoking: `CellularAutomataPhaseTransform` with `{width: 100, 'attach\_prob': 0.2, 'num\_solid\_states': 3, 'num\_nuclei': 5}` The simulation for the solidification of an Al-Cu-Mg alloy system using a Cellular Automata model on a 100x100 grid with an attachment probability of 0.2 has been initiated. The system is set to have 3 solid states and 5 initial nuclei for the transformation process.

> Finished chain.

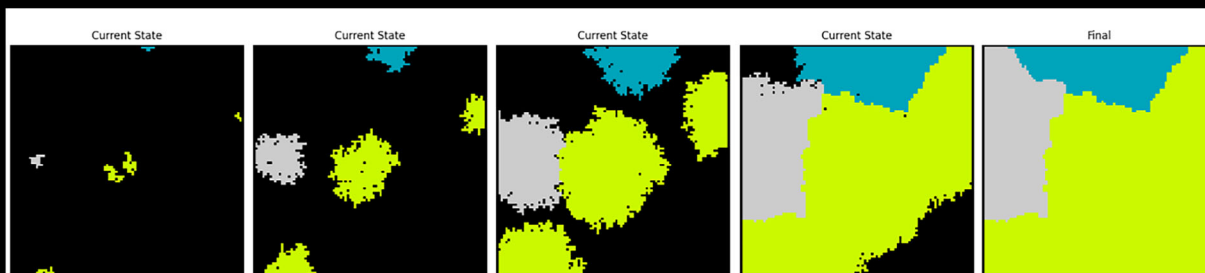


Fig. 6 | Cellular Automata simulation of diffusion-controlled growth. a User query specifying the simulation task. **b** Agent-generated output, including frame-by-frame microstructure evolution.

(a) User Query

Simulate the solidification of a Ti-Al intermetallic alloy using a Cellular Automata (CA) model. Utilize a grid size of 150x150 for a compact domain while ensuring sufficient resolution of microstructural features. Set the nucleation density to 50 nuclei per unit area to represent controlled nucleation. Model the attachment probability to favor interface controlled growth, promoting smooth, isotropic solidification fronts with minimal dendritic branching, characteristic of high attachment rates under rapid thermal gradients.

(b) Continuum Simulation Agent

> Entering new AgentExecutor chain...

Invoking: `CellularAutomataPhaseTransform` with `{width: 150, 'attach_prob':

0.9, 'num_solid_states': 2, 'num_nuclei': 50}` The simulation for the solidification of a Ti-Al intermetallic alloy using a Cellular Automata model with the specified parameters has been initiated. The grid size is set to 150x150, the attachment probability is 0.9 to favor interface-controlled growth, and there are 50 nuclei per unit area for controlled nucleation. This setup aims to promote smooth, isotropic solidification fronts with minimal dendritic branching.

> Finished chain.

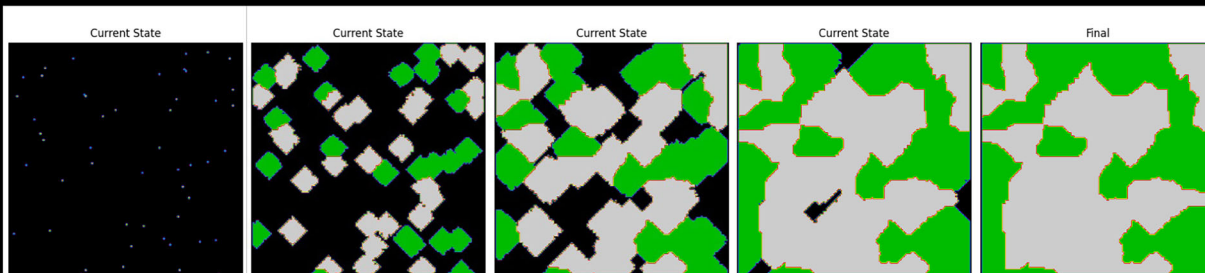


Fig. 7 | Cellular automata simulation of interface-controlled growth. a User query specifying the simulation task. **b** Agent-generated output, including frame-by-frame microstructure evolution.

(a) User Query

Using Monte Carlo annealing at $T \sim 0K$ simulate grain growth in a polycrystalline aluminum sample, to model the minimization of grain boundary energy during annealing. Use a 50×50 lattice grid, with each site representing a distinct grain orientation chosen from 20 possible flavors. Begin by initializing the lattice with random grain orientations and identifying grain boundaries. Focus updates on interface sites, where neighboring grains meet, to simulate energy reduction at grain boundaries. Perform 100 Monte Carlo steps, dynamically tracking and updating the positions of interface sites to reflect grain coarsening. Capture snapshots of the lattice at regular intervals to observe the evolution of grain structure, including the growth of larger grains at the expense of smaller ones.

(b) Continuum Simulation Agent

> Entering new AgentExecutor chain...

Invoking: `BiasedPottsMonteCarloAnnealing` with `{ 'Q': 20, 'L': 50, 'MC\_steps': 100, 'biasing\_strategy': 3 }`

> Finished chain.

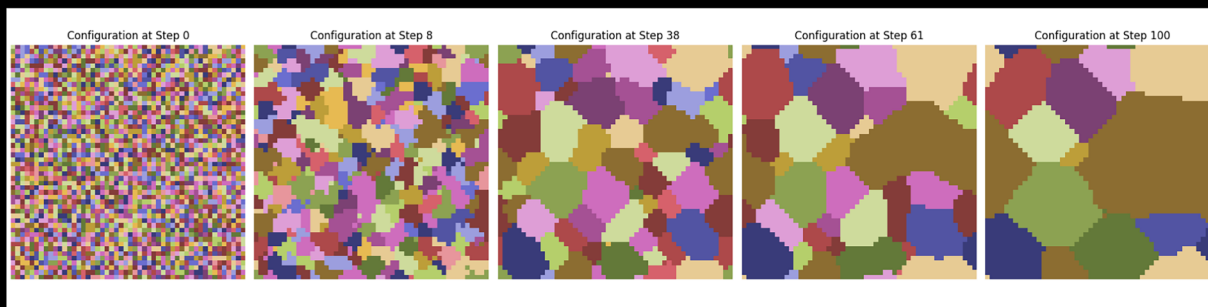


Fig. 8 | Monte Carlo Annealing simulation of polycrystalline grain growth. a User query specifying the simulation task. **b** Agent-generated output, including frame-by-frame microstructure evolution.

(a) User Query

Create a CIF file for an alloy with chemical formula Ni_3Al in the L12 crystal structure (space group: $Pm-3m$).

(b) Crystal Generation Agent

> Entering new AgentExecutor chain...

Invoking: `CrystalLM` with `{ 'composition': 'Ni3Al', 'sg': 'Pm-3m' }`

The CIF file provided contains information about a crystalline structure of $AlNi_3$. The structure belongs to the $Pm-3m$ space group and has a cubic unit cell with cell parameters $a=b=c=3.5555 \text{ \AA}$. The chemical formula of the structure is $AlNi_3$, with one formula unit per unit cell. The crystallographic data includes the atomic positions and occupancies, with Al atoms positioned at $(0,0,0)$ and Ni atoms at $(0,0.5,0.5)$. The file also provides details about the elemental properties such as electronegativity and ionic radius for Al and Ni atoms.

> Finished chain.

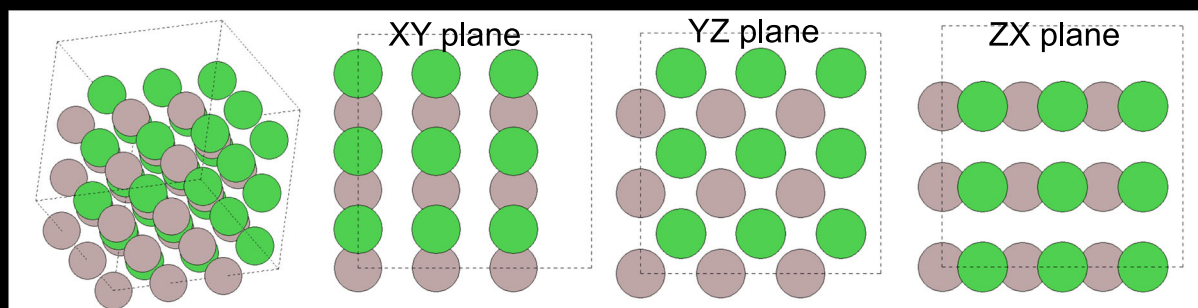


Fig. 9 | Crystallographic generation for Ni_3Al . a User query specifying the chemical composition and space group. **b** Agent-generated CIF file description and corresponding visualization of the crystal structure.

(a) User Query

Create a CIF file for a material with the chemical composition Sr₂FeMoO₆ in the double perovskite structure.

(b) Crystal Generation Agent

> Entering new AgentExecutor chain...

Invoking: `CrystaLLM` with `{ 'composition': 'Sr2Fe1Mo1O6' }`

The cif file provides data for the compound Sr₂FeMoO₆ with crystallographic information. The crystal structure belongs to space group P4/mmm with cell parameters a=b=5.65 Å, c=3.9733 Å, and angles alpha=beta=gamma=90°. The compound has a chemical formula Sr₂FeMoO₆, where Sr, Fe, Mo, and O occupy specific fractional coordinates in the unit cell. The atom types have different electronegativity and ionic radius values. This information can be used for further analysis and simulation of the compound's properties. However, there is a warning that the file was not post-processed properly by CrystaLLM, indicating potential issues with the data that need to be addressed before using it for in-depth investigations. The CIF file for the material with the chemical composition Sr₂FeMoO₆ in the double perovskite structure has been generated.

> Finished chain.

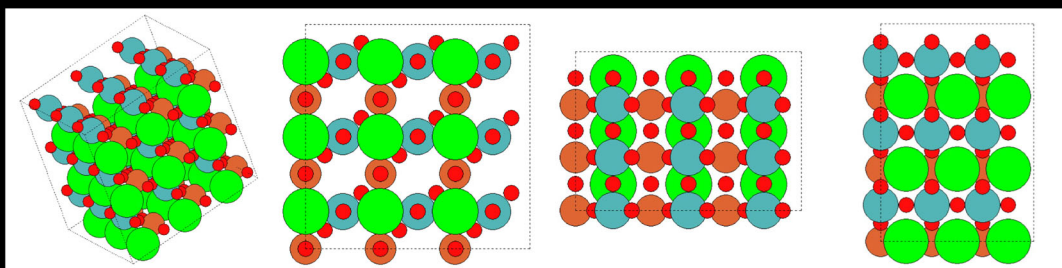


Fig. 10 | Crystallographic generation for Sr₂FeMoO₆. **a** User query specifying the chemical composition and application. **b** Agent-generated CIF file description and corresponding visualization of the crystal structure.

Table 1 | Common materials IDs across runs

	0	1	2	3	4	5	6	7	8	9
0	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
1	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
2	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
3	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
4	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
5	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
6	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
7	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
8	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0
9	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0

IoU Matrix of Material IDs Between Runs.

independently functioning agents. It also highlights the potential of using this agent framework to handle multiple tasks effectively.

Reliability analysis | Materials extraction

To evaluate the consistency of the LLM agent, we performed 10 independent runs of the same prompt. All runs selected: MaterialExtractionAgent and tools: get_material_ids and MaterialsProjectBonds. Among the 10 runs, runs 0–4 and 6–9 yielded identical sets of 47 material IDs. Run 5 failed to extract any material IDs, suggesting an isolated error. The consistency in the other nine runs demonstrates the stability and reproducibility of the material extraction component of the agent.

To gauge the similarity between the different materials ids that the agent was returning we decided to calculate the Intersection over Union (IoU) matrix between all the runs. This is basically the number of common material ids returned in the runs divided by the combined material ids across

both runs.

$$\text{IoU}_{ij} = \frac{|A_i \cap A_j|}{|A_i \cup A_j|}$$

where A_i is the set of material IDs returned in run i , A_j is the set of material IDs returned in run j , $A_i \cap A_j$ denotes the intersection (i.e., common material IDs) between runs i and j and $A_i \cup A_j$ denotes the union (i.e., total unique material IDs) across runs i and j .

The Intersection over Union (IoU) matrix of material ID sets across runs is shown in Table 1. An IoU of 1.0 indicates perfect overlap. As seen in the matrix, all runs except run 5 show perfect agreement (IoU = 1.0), while run 5 shows zero overlap (IoU = 0.0) with all others.

Minimum, maximum, and average bond lengths per run are listed in Fig. 13. Bond length values are consistently distributed across all successful runs.

The standard deviations for minimum, maximum and average bond lengths across all runs (excluding run 5) are $\sigma_{\min} = 0.05025$ Å, $\sigma_{\max} = 0.06833$ Å and $\sigma_{\text{avg}} = 0.05890$ Å. These low values confirm that bond length extraction is highly stable across runs. Every successful run consistently identified the same coordination patterns: $\text{O}^{2-}-\text{Ti}^{4+}$ and $\text{Ti}^{4+}-\text{O}^{2-}$.

To assess consistency in responses generated for materials synthesis queries 10 prompt executions are analyzed. Each response was logged and reviewed to evaluate agent/tool selection, compounds of focus, synthesis method association, output formatting, and level of detail. For all 10 responses MatSciAgent delegated the task to MaterialExtractionAgent and MPSynthesisTool which is specialized for extracting synthesis-related information. This demonstrated perfect consistency in agent and tool selection rationale. The most frequently extracted materials across runs are listed below, along with their frequencies and synthesis methods in Table 2. The basic contents of the response and their corresponding formats can be found in Table 3.

(a) User Query

Perform a molecular dynamics simulation of aluminum (Al) under constant energy (NVE) conditions using the Effective Medium Theory (EMT) potential. The system should be initialized with a face-centered cubic (FCC) crystal structure, using a lattice constant of 4.05. Set the temperature to 660 C and run the simulation for 100000 steps. Use the EMT potential to model interatomic interactions for aluminum.

(b) Molecular Dynamics Agent

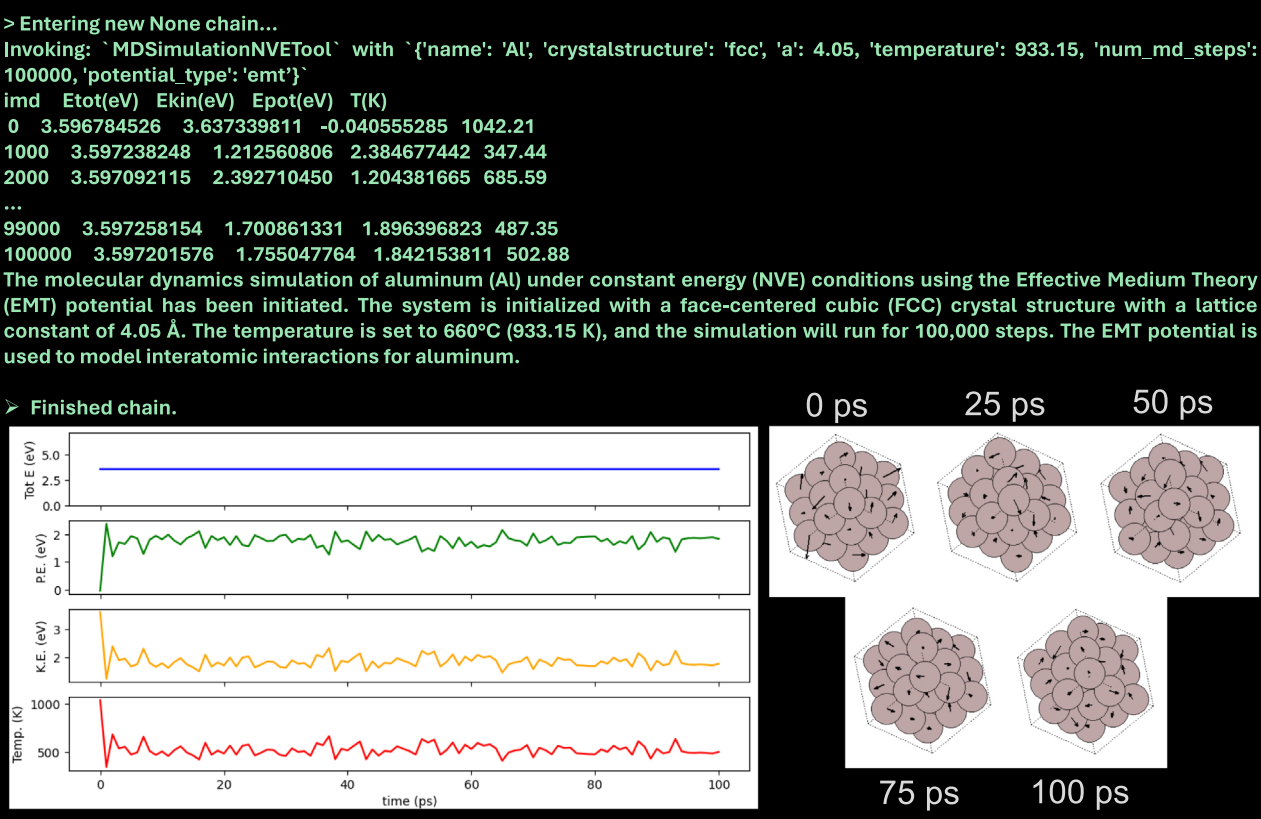


Fig. 11 | MD simulation of aluminum. **a** User query specifying the simulation conditions, including ensemble type and temperature. **b** Agent-generated output showing a summary of the simulation setup and the resulting MD trajectory.

Table 2 | Frequency of Extracted Compounds Across 10 Runs

Compound	Runs Found	Frequency	Run Indices	Synthesis Method
BiCuVO _x	6	60%	0, 1, 3, 6, 7, 8	Sol-Gel
Cu ₂ Zn(Sn)Se ₄	4	40%	2, 4, 5, 9	Solid-State
La _{0.8} Sr _{0.2} Co _x Fe _{1-x} O ₃	3	30%	4, 5, 9	Solid-State
La _{0.6} Sr _{0.4} Mo _{0.2} Fe _{0.8} O _{3-δ}	5	50%	1, 3, 6, 7, 8	Sol-Gel
SrFeO _{3-δ}	4	40%	0, 1, 6, 7	Solid-State
(K _x Na _{1-x})NbO ₃	6	60%	0, 1, 3, 6, 7, 8	Sol-Gel
BaZr _{0.1} Ce _{0.7} Y _{0.2} O ₃	6	60%	0, 1, 3, 6, 7, 8	Solid-State
Al _x Zn _{1-x} O	2	20%	0, 6	Sol-Gel

This table displays analysis of the compounds and their methods returned during consistency analysis.

Reliability analysis | Continuum simulation

We tested reproducibility for three continuum simulation prompts by executing each prompt five times. The simulations were:

1. Al-Cu-Mg Alloy Solidification (Cellular Automata)
2. Ti-Al Intermetallic Alloy Solidification (Cellular Automata)

3. Polycrystalline Aluminum Grain Growth (Monte Carlo Annealing)

The agent selection and tool invocation were fully deterministic across all runs.

ContinuumSimulationAgent was consistently selected by MatSciAgent across all prompts and all runs. For the Al-Cu-Mg and Ti-Al solidification prompts, the agent invoked the CASolidTool (CellularAutomataPhaseTransform), suitable for cellular automata-based simulations. For the polycrystalline aluminum grain growth prompt, the agent invoked the MCAnnealingTool (BiasedPottsMonteCarloAnnealing), which is designed for Monte Carlo annealing simulations of grain growth.

Across all runs, key simulation parameters including grid dimensions, attachment probability, and number of nuclei or phases were consistently interpreted and as shown in Table 4. No discrepancies were observed in the parsed values. Additionally, the agent reliably mapped each simulation prompt to the correct modeling technique (Cellular Automata or Monte Carlo Annealing), regardless of run index.

Reliability analysis | MD simulation

To evaluate the robustness and reproducibility of our agent framework, we performed a consistency analysis by executing the same MD simulation prompt ten times. This analysis assesses the agent’s ability to consistently select the correct agent, extract parameters, invoke tools, and execute the simulation, as illustrated in Table 5. All 10 runs correctly selected the

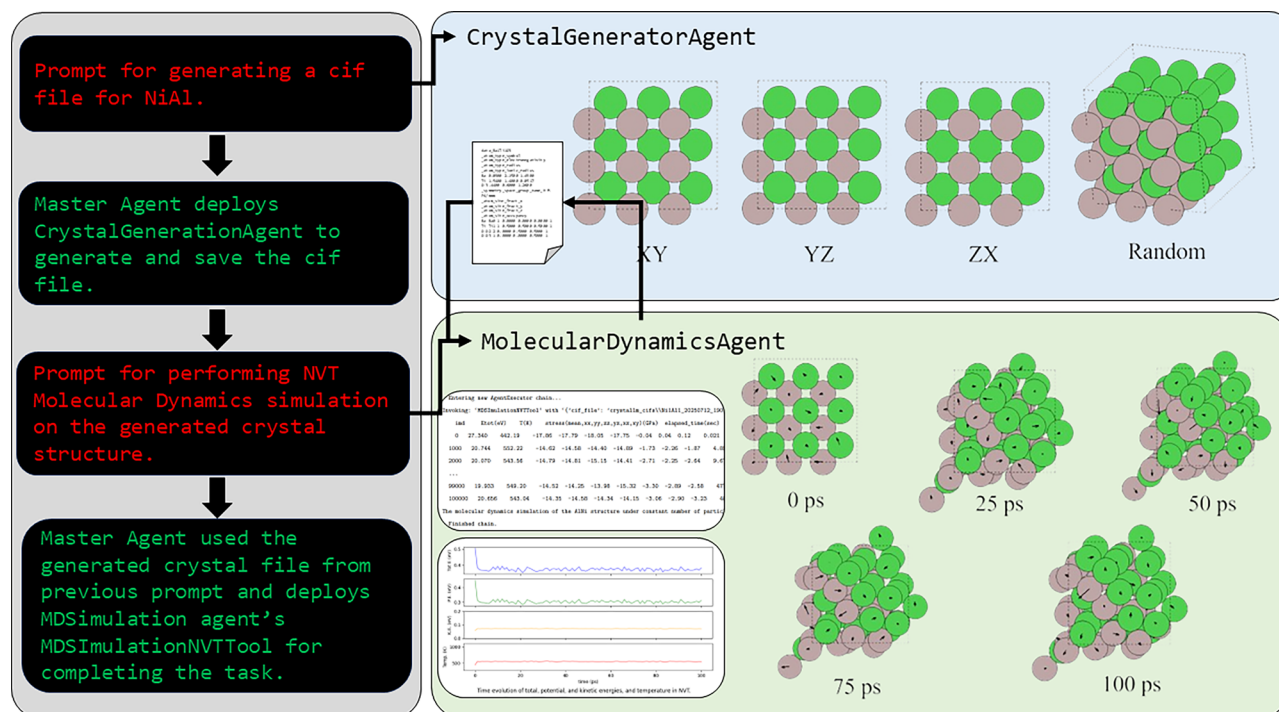


Fig. 12 | Coordinating multiple agents for joint task. After generating the CIF file the master agent successfully deployed MolecularDynamicsAgent to use the cif file generated previously to run the MD simulation.

MolecularDynamicsAgent as the appropriate sub-agent to handle the simulation task. In every run, the agent successfully extracted all 8 key simulation parameters and invoked the MDSimulationNVTTool with an identical dictionary:

```
{'name': 'Al',
'crystalstructure': 'fcc',
'a': 4.05,
'cubic': True,
'potential_type': 'emt',
'temperature': 300,
'num_md_steps': 10000,
'time_step': 1}
```

Each run executed for 10,000 steps with energy conservation maintained across the simulation. Minor variation in physical outputs, such as energy and final temperature, were observed as expected where the initial total energy ranged from 3.63 eV to 4.29 eV and final temperatures ranged from ~130 K–180 K.

The results highlight the high reliability and reproducibility of the agent framework when tasked with molecular dynamics simulations. The agent consistently selected the correct tools, interpreted parameters without error, and completed valid simulations under NVE conditions. The observed variability in thermodynamic outputs is physically expected due to stochastic effects inherent to MD. Minor variations in the presentation style do not affect the scientific correctness or repeatability of the simulation process.

Conclusion

The integration of LLMs with simulation tools demonstrates their growing utility in computational materials science. This paper showcases their effectiveness in structure retrieval and generation, as well as in running continuum and MD simulations. The case studies highlight the flexibility of LLMs in automating workflows that incorporate both custom code and established software. The agent consistently extracts relevant

parameters and search keywords from natural language input with high accuracy.

The reliability analysis demonstrates that the framework maintains consistent agent and tool selection, parameter extraction, and simulation execution across repeated runs. With near-perfect reproducibility in most tasks and only minor, non-impacting variations in formatting or isolated extraction failures, the system shows the stability required for materials research.

A key strength of the framework is its modular design, which allows it to expand as new tools are added. Broadening data sources beyond the Materials Project and MatWeb would further enhance its applicability. Simulation capabilities could be improved through advanced workflows, multiscale modeling, and ML-guided optimization. Moreover, enabling the agent to dynamically write or adapt code would increase its flexibility, making it better suited for unseen or specialized tasks.

Methods

Language model

We utilize OpenAI's ChatGPT API, specifically the gpt-3.5-turbo model configured with temperature value of 0, to support natural language understanding and generation within the MatSciAgent framework. This configuration was applied uniformly across all agent types (Master, Material Extraction, Continuum Simulation, Crystal Generation, and Molecular Dynamics) to ensure consistent query routing and tool execution. GPT-3.5-turbo is part of the Generative Pretrained Transformer (GPT) series⁴³, which is based on the transformer architecture and leverages self-attention mechanisms⁴⁴ to model complex language patterns and dependencies. While GPT-3.5-turbo is used in our current implementation due to its favorable balance between performance and cost, the modular design of our framework allows for easy substitution with more advanced models, depending on task requirements or resource availability.

To manage the interactions between the language model and the system modules, we use the LangChain framework⁴⁵. LangChain provides tools for prompt management, output parsing, and tool integration, enabling structured communication and orchestration between the language model and external APIs or computational components. As of this

Fig. 13 | Spread of bond lengths across 9 runs. Box plot of minimum, maximum, and average atomic distances across 9 successful simulation runs. The narrow deviation between the runs demonstrates that the agent consistently produces stable and reproducible outputs.

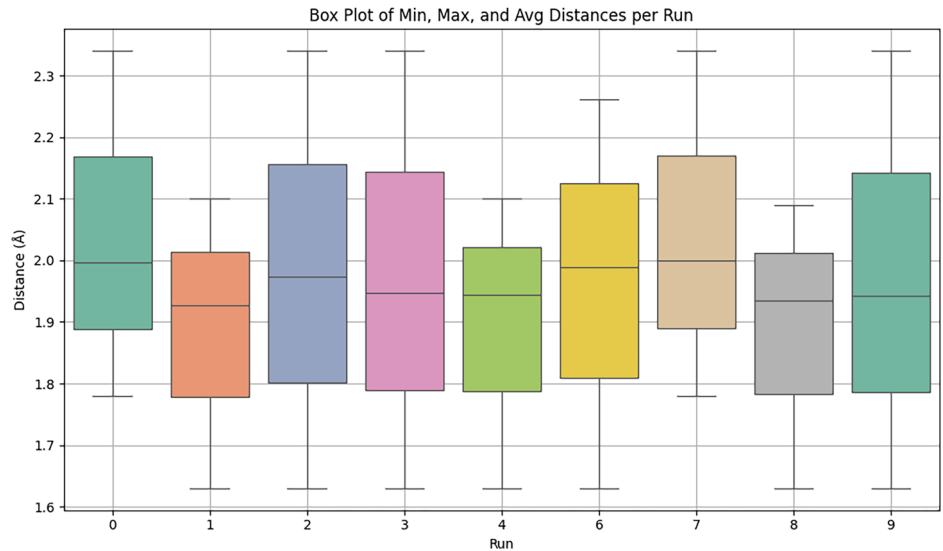


Table 3 | Synthesis Output Characteristics Across Agent Runs

Run	Format Type	Structure	Reaction Eq.	Temp.	Duration	Atmosphere
0	Bullet Points	Nested	✓	✓	✓	
1	Paragraph	Continuous	✓	✓	✓	✓
2	Bullet Points	Nested	✓	✓	✓	✓
3	Bullet Points	Nested	✓	✓	✓	
4	Paragraph	Structured	✓	✓	✓	✓
5	Paragraph	Continuous	✓	✓	✓	✓
6	Paragraph	Continuous	✓	✓	✓	✓
7	Paragraph	Continuous	✓	✓	✓	
8	Bullet Points	Flat	✓	✓	✓	
9	Paragraph	Structured	✓	✓	✓	✓

This table shows the response format that the broad synthesis details included in each output during reliability analysis.

Table 4 | Consistency of agent and tool selection

Solidification Al-Cu-Mg Alloy	Grain Growth via Monte Carlo Annealing	Solidification Ti-Al Intermetallic Alloy
Agent: ContinuumSimulationAgent	Agent: ContinuumSimulationAgent	Agent: ContinuumSimulationAgent
Tool: CASolidTool	Tool: MCAnnealingTool	Tool: CASolidTool
Parameters: {"width": 100, "attach_prob": 0.2, "num_solid_states": 3, "num_nuclei": 5}	Parameters: {"Q": 20, "L": 50, "MC_steps": 100, "biasing_strategy": 3}	Parameters: {"width": 150, "attach_prob": 0.9, "num_solid_states": 2, "num_nuclei": 50}

This table shows the repetition stability of the agent across three continuum simulation tasks (5 runs each)

writing, the API cost for gpt-3.5-turbo is \$0.0015 per 1,000 input tokens and \$0.002 per 1,000 output tokens (<https://openai.com/pricing>), making it a practical choice for iterative, multi-turn interactions in materials science workflows.

Materials database

The Materials Project is an open-access database designed to support computational materials science research³. It contains over 170,000 material entries, all generated using first-principles density functional theory (DFT) calculations. The database offers a comprehensive set of material properties, including crystal structures, formation energies, band structures, density of

states, elastic tensors, bulk and shear moduli, Poisson’s ratio, dielectric constants, magnetic properties, phonon dispersion (for selected materials), and surface energies. The dataset is continuously updated, and users can access it programmatically via a well-documented API, commonly through tools like pymatgen.

MatWeb is an online database that focuses on experimentally measured or manufacturer-provided material properties⁴. It features over 180,000 material datasheets, covering a wide variety of materials such as metals, alloys, polymers, ceramics, composites, and glasses. MatWeb provides detailed information on mechanical properties (e.g., tensile strength, yield strength, hardness), thermal properties (e.g., thermal conductivity,

Table 5 | Reliability analysis for Molecular dynamics agent

Metric	Consistency	Notes
Agent Selection	100%	Always selected <code>MolecularDynamicsAgent</code>
Parameter Extraction	100%	All 8 key parameters extracted correctly
Tool Invocation	100%	Identical calls to <code>MDSimulationNVETool</code>
Simulation Execution	100%	All completed 10,000 steps
Output Format	80%	Minor variations in style (bullet vs paragraph)
Content Accuracy	100%	All essential information was included

Quantitative Consistency Metrics Across 10 Runs for molecular dynamics reliability analysis.

expansion coefficients), electrical properties, chemical compositions, and physical constants like density and specific heat.

Materials extraction agent

LLMs, while powerful, are typically trained on general-purpose natural language datasets and lack access to the specialized, structured data required for materials science applications^{46,47}. Because they rely on static, pre-trained knowledge, they cannot dynamically query authoritative databases, often resulting in outdated or incomplete information. Furthermore, their probabilistic nature can lead to overgeneralization or misinterpretation of complex scientific queries⁴⁸.

To overcome these limitations, we introduce a materials extraction agent that integrates the Materials Project API and custom web-scraping for MatWeb, enabling real-time access to validated, domain-specific data. Upon receiving a user query, the agent retrieves relevant material properties—such as thermal, dielectric, and surface characteristics—by invoking the appropriate database call, either via API or web scraping. It then synthesizes a natural language response based on the retrieved data. This method ensures that responses are grounded in current, authoritative sources, allowing for more accurate contextualization of materials, verification of properties under specific conditions, and delivery of reliable, data-driven insights critical for real-world applications. Importantly, by relying on scientifically validated sources, the system also enhances the reproducibility of output prompt generation.

The agent retrieves data from structured databases, including the Materials Project³ and MatWeb⁴. The Materials Project offers access to over 150,000 materials with properties such as elasticity, phonon spectra, and thermodynamic stability—essential for predictive modeling. MatWeb complements this with experimentally validated material data, enabling direct comparisons between computational predictions and empirical behavior. Further implementation details are provided in the Methods section.

Cellular automata

In this work, the Cellular Automata model is used to simulate phase transformations such as solidification or recrystallization within a two-dimensional, grid-based system. Each cell in the grid represents a point in space and can exist either in the liquid phase, denoted by 0, or in one of several solid phases, represented by positive integers. The transformation process is driven by local interactions between neighboring cells and governed by an attachment probability, allowing the model to effectively capture the evolution of microstructures during phase changes.

The model is defined by several key parameters. The `width` specifies the size of the square simulation grid, and the simulation is restricted to the interior cells to avoid boundary effects. The `attach_prob` parameter controls the likelihood that a liquid cell will transform into a solid phase based on its neighbors. When this value is set to 1, the transformation is deterministic, while lower values introduce randomness, resulting in

probabilistic growth behavior. The `num_solid_states` parameter determines how many distinct solid phases can exist in the system. When set to 1, the model simulates basic solidification with a single growing phase; when >1 , it enables more complex recrystallization scenarios, where multiple phases grow and compete for space.

Nucleation is introduced through the `place_nuclei(num_nuclei)` function, which randomly places a specified number of initial solid cells throughout the grid. Each nucleus is assigned a random solid phase. The system then evolves over a series of discrete time steps using the `evolve(Nsteps)` function, which updates each cell based on the states of its neighbors.

The output of the simulation includes visual snapshots of the evolving microstructure at each time step, as well as a plot showing the fraction of the transformed area over time. This transformation curve can be compared to analytical predictions based on the Johnson-Mehl-Avrami-Kolmogorov (JMAK) theory⁴⁹, providing a means of validating the model's behavior.

Monte Carlo annealing

Monte Carlo Annealing is used to simulate grain growth or spin ordering in a two-dimensional Potts model at very low temperatures ($T \approx 0$), where the system dynamics are governed primarily by energy minimization⁵⁰. This corresponds to zero-temperature Metropolis dynamics, in which spin flips are only accepted if they lower or maintain the system's energy. Each site on the 2D lattice can exist in one of Q possible states, and the system evolves by locally minimizing interfacial energy between different states.

The model is characterized by several key parameters. The variable Q defines the number of possible states or “flavors” per lattice site, which can represent, for example, distinct grain orientations in a polycrystalline material. The parameter L sets the size of the square lattice, resulting in a total of $L \times L$ sites. The simulation proceeds for a given number of Monte Carlo steps, with each step consisting of L^2 spin-flip attempts. The biasing strategy determines how both the site and the trial state are selected during simulation. Strategies include selecting a random new state from all Q options, choosing from neighboring states at any site, or restricting to neighboring states only at interfacial sites (i.e., those bordering unlike neighbors).

This simulation produces several outputs that help analyze the system's behavior. The energy history tracks how the total energy of the lattice evolves over time, typically decreasing as the system anneals. The acceptance history records the number of accepted spin flips per Monte Carlo step, providing insight into how active or “frozen” the system becomes during the simulation. The configuration history stores snapshots of the lattice at each step, allowing visualization of the evolving microstructure and phase ordering. If plotting is enabled, the simulation also generates visualizations of the energy curve, acceptance rate, and both initial and final configurations. Additionally, it can save all configuration snapshots as images in a results directory for further analysis.

Continuum simulation agent

The continuum simulation agent executes advanced materials science simulations, integrating two key modeling techniques: Cellular Automata (CA)^{51,52} for phase transformations and Monte Carlo Annealing (MCA)^{50,53} for microstructural evolution during annealing. Users provide simulation conditions in natural language, which the agent translates into actionable parameters before executing the simulation.

The CA tool models phase transformations such as solidification and recrystallization by iteratively updating a 2D grid based on local neighborhood conditions. It accounts for attachment probabilities and tracks transformed regions over time, offering insights into grain growth and nucleation in both single-phase and multiphase systems.

MCA tool, using the Potts model⁵⁴, simulates the evolution of the grain structure under annealing conditions. Operating near zero temperature, it employs three biasing strategies: unbiased selection, neighboring flavor

selection, and interface bias selection to model interface and bulk site behaviors. This method enables a detailed analysis of grain boundary dynamics and energy minimization during thermal processing, offering predictive insights into microstructural stability.

Understanding phase transformations and microstructural evolution is essential for tailoring material properties in manufacturing and processing. By integrating CA and MCA within an LLM framework, it enhances traditional modeling tools with natural language interaction, parameter customization, and visualization. It delivers actionable insights through simulation results.

CrystaLLM

CrystaLLM is a domain-specific LLM designed to generate crystallographic structures in CIF format from basic compositional input. Built on the GPT-2 architecture and adapted using the nanoGPT implementation^{55,56}, CrystaLLM has been purposefully trained for applications in inorganic materials science. Rather than relying on general-purpose vocabularies, it uses a custom byte-level tokenizer with a vocabulary of 371 tokens tailored to the syntactic patterns of CIF files.

The training dataset for CrystaLLM was assembled from multiple high-quality materials science databases, including the Materials Project³, the Open Quantum Materials Database⁵, and the NOMAD⁵⁷. These sources provide a diverse range of inorganic crystal structures, enabling the model to learn detailed patterns related to atomic arrangements, symmetry operations, and lattice parameters across a wide variety of crystal systems.

We employ the small variant of CrystaLLM, with temperature set to 0.8; top-k sampling with $k = 10$; random seed fixed at 1337 for reproducibility; executed with bfloat16 in our study. This version consists of 8 transformer layers, each with 8 attention heads and a context window of 1,024 tokens. The model has an embedding dimension of 512 and was trained from scratch for 5,000 iterations.

Crystal structure generation agent

Atomic structure is fundamental to understanding inorganic crystal materials and performing computational simulations. When structural data is available in existing datasets, it can be directly retrieved. However, if such data is missing, alternative methods are needed to generate the desired atomic structures.

The crystal generator agent addresses this by generating crystal structures based on elemental compositions or alloy systems using CrystaLLM, a transformer-based language model specifically designed to produce CIFs (Crystallographic Information Files) of inorganic crystals⁸. Upon receiving a user query, the agent extracts relevant information and runs CrystaLLM to generate plausible inorganic crystal structures. It can produce multiple candidate structures for a given composition, enabling material exploration.

Unlike general-purpose LLMs, CrystaLLM is tailored to recognize complex crystallographic patterns, enabling the generation of physically realistic structures. As part of a multi-agent system, the crystal generator agent streamlines materials workflows by providing immediate input for downstream tasks such as thermodynamic stability analysis, property prediction, and MD simulations. While CrystaLLM is presented here as an example generative model for crystal structure generation based on composition and space group information note that the framework is modular and can accommodate additional generative models, including those capable of property-constrained generation, allowing future extensions toward inverse materials design as well.

Molecular dynamics simulation

MD simulations were conducted using the Atomic Simulation Environment (ASE) to investigate the structural evolution of atomic systems under various thermodynamic conditions. The agentic framework enables the execution of MD simulations in NVE (constant energy), NVT (constant Number, Volume, Temperature), and NPT (constant Number, Pressure, Temperature) ensembles. Thermodynamic control was achieved using the

Berendsen thermostat for temperature regulation and the Berendsen barostat for pressure regulation in NPT simulations.

Key simulation parameters include the time step for numerical integration, target temperature, and the total number of MD steps. For ensemble control, the thermostat and barostat time constants were specified, along with external pressure and barostat coupling strength for NPT conditions. Additionally, a logging interval was defined to record system properties and trajectories at regular time steps. Simulation outputs consist of atomic trajectories saved in `.traj` format, log files detailing the evolution of energy, temperature, and stress, and corresponding visualizations of energy and temperature profiles over time.

Molecular dynamics agent

Atomic-scale simulations are essential for understanding fundamental material behaviors, such as thermal conductivity, mechanical strength, and phase transitions^{58,59}. The MD simulation agent is designed to automate and streamline atomic-scale simulations. It interprets user input queries, sets up the appropriate simulation parameters, and executes the MD simulation accordingly. Specifically, the agent initializes atomic structures, defines interatomic potentials, and runs simulations using tools primarily based on the Atomic Simulation Environment (ASE) package⁶⁰.

It supports the construction of atomic structures with various crystal symmetries and accommodates a range of interatomic potentials, including the Embedded Atom Method (EAM)⁶¹, Lennard-Jones (LJ)⁶², and Effective Medium Theory (EMT)⁶³. These enable simulations of metals, alloys, and simple molecular systems. The agent can also perform simulations under different thermodynamic ensembles—NVE (constant energy), NVT (constant temperature), and NPT (constant pressure)—allowing users to explore material behavior under controlled conditions. Further details are provided in the Methods section. By translating natural language input into simulation-ready parameters, the agent simplifies the MD simulation workflow.

Integrating the MD simulation tools with an LLM agent significantly enhances its capabilities. Users can interactively configure simulations, analyze results, and visualize atomic trajectories using natural language queries. Unlike traditional standalone MD tools, this agent automates the entire workflow—from structure initialization to result interpretation—providing actionable outputs such as energy plots and trajectory data with minimal manual effort.

Ethics and inclusion

This study uses only publicly available data and adheres to ethical research standards. Code and methodology will be shared to promote transparency, reproducibility, and collaboration.

Technology use disclosure

ChatGPT was utilized for grammar and spelling corrections during manuscript preparation. All authors have reviewed and verified the accuracy of the final content.

Data availability

The case study results and reliability study results are provided as supplementary data.

Code availability

The necessary code used in this study can be accessed at <https://github.com/cakshat/MatSci-LLM-Agents>.

Received: 5 May 2025; Accepted: 14 October 2025;

Published online: 26 March 2026

References

1. Pollice, R. et al. Data-driven strategies for accelerated materials design. *Acc. Chem. Res.* **54**, 849–860 (2021).

2. Kalidindi, S. R. & De Graef, M. Materials data science: current status and future outlook. *Annu. Rev. Mater. Res.* **45**, 171–193 (2015).
3. Jain, A. et al. The materials project: a materials genome approach to accelerating materials innovation. *APL Mater.* **1**, 011002 (2013).
4. LLC, M. *Matweb: Online Materials Information Resource*. <http://www.matweb.com> (2020).
5. Kirklin, S. et al. The open quantum materials database (oqmd): assessing the accuracy of DFT formation energies. *npj Comput. Mater.* **1**, 1–15 (2015).
6. Hellenbrandt, M. The inorganic crystal structure database (ICSD)-present and future. *Crystallogr. Rev.* **10**, 17–22 (2004).
7. Tanaka, I., Rajan, K. & Wolverton, C. Data-centric science for materials innovation. *MRS Bull.* **43**, 659–663 (2018).
8. Antunes, L. M., Butler, K. T. & Grau-Crespo, R. Crystal structure generation with autoregressive large language modeling. *Nat. Commun.* **15**, 10570 (2024).
9. Xie, T., Fu, X., Ganea, O.-E., Barzilay, R. & Jaakkola, T. Crystal diffusion variational autoencoder for periodic material generation. *arXiv* <https://arxiv.org/abs/2110.06197> (2022).
10. Zeni, C. et al. A generative model for inorganic materials design. *Nature* **639**, 624–632 (2025).
11. Himanen, L., Geurts, A., Foster, A. S. & Rinke, P. Data-driven materials science: status, challenges, and perspectives. *Adv. Sci.* **6**, 1900808 (2019).
12. Vaswani, A. et al. Attention is all you need. In Guyon, I. et al. (eds.) *Advances in Neural Information Processing Systems*, vol. 30 (Curran Associates, Inc., 2017).
13. Brown, T. et al. Language models are few-shot learners. *Adv. neural Inf. Process. Syst.* **33**, 1877–1901 (2020).
14. Achiam, J. et al. *Gpt-4 Technical Report*. <https://cdn.openai.com/papers/gpt-4.pdf> (2023).
15. Touvron, H. et al. Llama: Open and efficient foundation language models. *arXiv* <https://arxiv.org/abs/2302.13971> (2024).
16. Anthropic. *Claude 3.7 Sonnet*. <https://www.anthropic.com> (2025).
17. Tshitoyan, V. et al. Unsupervised word embeddings capture latent knowledge from materials science literature. *Nature* **571**, 95–98 (2019).
18. Ock, J., Guntuboina, C. & Barati Farimani, A. Catalyst energy prediction with CatBERTA: unveiling feature exploration strategies through large language models. *ACS Catal.* **13**, 16032–16044 (2023).
19. Ock, J., Badrinarayanan, S., Magar, R., Antony, A. & Barati Farimani, A. Multimodal language and graph learning of adsorption configuration in catalysis. *Nat. Mach. Intell.* **6**, 1501–1511 (2024).
20. Cao, Z., Magar, R., Wang, Y. & Barati Farimani, A. Moformer: Self-supervised transformer model for metal-organic framework property prediction. *J. Am. Chem. Soc.* **145**, 2958–2967 (2023).
21. Xu, C., Wang, Y. & Barati Farimani, A. Transpolymer: a transformer-based language model for polymer property predictions. *npj Comput. Mater.* **9**, 64 (2023).
22. Kim, S., Mollaei, P., Antony, A., Magar, R. & Barati Farimani, A. Gpcrbert: Interpreting sequential design of g protein-coupled receptors using protein language models. *J. Chem. Inf. Modeling* **64**, 1134–1144 (2024).
23. Mollaei, P., Sadasivam, D., Guntuboina, C. & Barati Farimani, A. Idp-bert: Predicting properties of intrinsically disordered proteins using large language models. *J. Phys. Chem. B* **128**, 12030–12037 (2024).
24. Badrinarayanan, S., Guntuboina, C., Mollaei, P. & Barati Farimani, A. Multi-peptide: Multimodality leveraged language-graph learning of peptide properties. *J. Chem. Inf. Model.* **65**, 83–91 (2025).
25. Meda, R. S. & Farimani, A. B. Bapulm: Binding affinity prediction using language models. *arXiv* <https://arxiv.org/abs/2411.04150> (2024).
26. Chaudhari, A., Guntuboina, C., Huang, H. & Farimani, A. B. Alloybert: Alloy property prediction with large language models. *Comput. Mater. Sci.* **244**, 113256 (2024).
27. Yeh, Y.-T., Ock, J. & Farimani, A. B. Text to band gap: Pre-trained language models as encoders for semiconductor band gap prediction. *arXiv* <https://arxiv.org/abs/2501.03456> (2025).
28. Pak, P. & Farimani, A. B. Additivellm: Large language models predict defects in additive manufacturing. *arXiv* <https://arxiv.org/abs/2501.17784> (2025).
29. Lu, C. et al. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv* <https://arxiv.org/abs/2408.06292> (2024).
30. Ock, J., Vinchurkar, T., Jadhav, Y. & Farimani, A. B. Adsorb-agent: Autonomous identification of stable adsorption configurations via large language model agent. *arXiv* <https://arxiv.org/abs/2410.16658> (2024).
31. Boiko, D. A., MacKnight, R., Kline, B. & Gomes, G. Autonomous chemical research with large language models. *Nature* **624**, 570–578 (2023).
32. Bran, A. M. et al. Augmenting large language models with chemistry tools. *Nat. Mach. Intell.* **6**, 525–535 (2024).
33. Jadhav, Y., Pak, P. & Farimani, A. B. LLM-3D print: Large language models to monitor and control 3d printing. *arXiv* <https://arxiv.org/abs/2408.14307> (2024).
34. Jadhav, Y. & Farimani, A. B. Large language model agent as a mechanical designer. *arXiv* <https://arxiv.org/abs/2404.17525> (2024).
35. Huang, L. et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv* <https://doi.org/10.1145/3703155> (2024).
36. Perković, G., Drobnjak, A. & Botički, I. Hallucinations in llms: Understanding and addressing challenges. In *2024 47th MIPRO ICT and Electronics Convention (MIPRO) 2084–2088* (IEEE, 2024).
37. Liu, F. et al. Exploring and evaluating hallucinations in LLM-powered code generation. *arXiv* <https://arxiv.org/abs/2404.00971> (2024).
38. Birhane, A., Kasirzadeh, A., Leslie, D. & Wachter, S. Science in the age of large language models. *Nat. Rev. Phys.* **5**, 277–280 (2023).
39. Johnson, S. & Hyland-Wood, D. A primer on large language models and their limitations. *arXiv* <https://arxiv.org/abs/2412.04503> (2024).
40. Chiang, Y., Hsieh, E., Chou, C.-H. & Riebesell, J. Llampt: Large language model made powerful for high-fidelity materials knowledge retrieval and distillation. *arXiv* <https://arxiv.org/abs/2401.17244> (2024).
41. Zhang, H., Song, Y., Hou, Z., Miret, S. & Liu, B. Honeycomb: A flexible LLM-based agent system for materials science. *arXiv* <https://arxiv.org/abs/2409.00135> (2024).
42. Ghafarollahi, A. & Buehler, M. J. Atomagents: Alloy design and discovery through physics-aware multi-modal multi-agent artificial intelligence. *arXiv* <https://arxiv.org/abs/2407.10022> (2024).
43. Radford, A. & Narasimhan, K. *Improving Language Understanding by Generative Pre-training*. <https://api.semanticscholar.org/CorpusID:49313245> (2018).
44. Vaswani, A. Attention is all you need. In *31st Conference on Neural Information Processing Systems*. (NIPS, 2017).
45. Chase, H. *Langchain*. <https://github.com/langchain-ai/langchain> (2022).
46. Ling, C. et al. Domain specialization as the key to make large language models disruptive: a comprehensive survey. *arXiv* <https://arxiv.org/abs/2305.18703> (2024).
47. Li, H. et al. Blade: Enhancing black-box large language models with small domain-specific models *arXiv* <https://arxiv.org/abs/2403.18365> (2024).
48. Mittelstadt, B., Wachter, S. & Russell, C. To protect science, we must use LLMs as zero-shot translators. *Nat. Hum. Behav.* **7**, 1830–1832 (2023).
49. Fanfoni, M. & Tomellini, M. The Johnson-Mehl-Avrami-Khokhlov model: a brief review. *Il Nuovo Cim. D.* **20**, 1171–1182 (1998).

50. Kinoshita, M., Okamoto, Y. & Hirata, F. First-principle determination of peptide conformations in solvents: Combination of Monte Carlo simulated annealing and RISM theory. *J. Am. Chem. Soc.* **120**, 1855–1863 (1998).
51. Wolfram, S. Statistical mechanics of cellular automata. *Rev. Mod. Phys.* **55**, 601 (1983).
52. Wolfram, S. Cellular automata as models of complexity. *Nature* **311**, 419–424 (1984).
53. Metropolis, N. & Ulam, S. The Monte Carlo method. *J. Am. Stat. Assoc.* **44**, 335–341 (1949).
54. Saito, Y. & Enomoto, M. Monte Carlo simulation of grain growth. *ISIJ Int.* **32**, 267–274 (1992).
55. Radford, A. et al. *Language Models are Unsupervised Multitask Learners*. <https://api.semanticscholar.org/CorpusID:160025533> (2019).
56. Karpathy, A. *NanoGPT*. <https://github.com/karpathy/nanoGPT> (2022).
57. Nomad: A distributed web-based platform for managing materials science research data. *J. Open Source Softw.* **8**, 5388 (2023).
58. Mavromaras, A. et al. Computational materials engineering: capabilities of atomic-scale prediction of mechanical, thermal, and electrical properties of microelectronic materials. In *2010 11th International Thermal, Mechanical & Multi-Physics Simulation, and Experiments in Microelectronics and Microsystems (EuroSimE), Bordeaux*, 1–10 (IEEE, 2010).
59. Eyert, V. et al. Atomistic simulations of microelectronic materials: prediction of mechanical, thermal, and electrical properties. In *Molecular Modeling and Multiscale Issues for Electronic Material Applications*. (eds. Iwamoto, N., Yuen, M., Fan, H.) (Springer, 2012).
60. Larsen, A. H. et al. The atomic simulation environment—a Python library for working with atoms. *J. Phys. Condens. Matter* **29**, 273002 (2017).
61. Foiles, S., Baskes, M. & Daw, M. S. Embedded-atom-method functions for the fcc metals Cu, Ag, Au, Ni, Pd, Pt, and their alloys. *Phys. Rev. B* **33**, 7983 (1986).
62. Lennard-Jones, J. E. On the forces between atoms and ions. *Proc. A* **109**, 584–597 (1925).
63. Choy, T. C. *Effective Medium Theory: Principles and Applications* 2nd edn, Vol. 256 (Oxford University Press, 2015).

Acknowledgements

The authors gratefully acknowledge support from the H. Robert Sharbaugh Presidential Fellowship.

Author contributions

A.C. conceived the study, developed the methodology, implemented the software, curated data, performed analysis, prepared visualizations, and

wrote the original draft. J.O. contributed to methodology, validation, investigation, data curation, and writing and editing. A.B.F. supervised the project, contributed to conceptualization, project administration, funding acquisition, and writing and editing.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s43246-025-00994-x>.

Correspondence and requests for materials should be addressed to Janghoon Ock or Amir Barati Farimani.

Peer review information *Communications Materials* thanks Federico Ottomano and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. A peer review file is available.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026